

概念板块波谷追踪 + 数据架构重构 — 设计文档 v1

1. 背景与目标

1.1 核心洞察

3L 体系的个股买卖点判断（L3）已经成熟，但在**板块层面**（L1/L2）的节奏判断存在断层：

当前系统：

大盘波峰波谷 → V5评分（pk_score 0~4）✔

板块排名 → 20日涨幅三梯队（主线/次级/非主线）✔

板块级别波谷机会 → 不存在 ✕

核心想法： 每个概念板块有自己的波谷位置。找到板块级别的波谷介入机会，比只看大盘更精准。板块从高位跌到谷底的过程，可能就是新介入机会的酝酿期。

1.2 行业板块 vs 概念板块

	行业板块（90个）	概念板块（399~486个）
分类依据	公司主营业务（申万/THS）	市场共识的叙事/主题
与逻辑的关系	弱	强（直接对应L2最强逻辑）
变化速度	慢	快（随市场热点增减）
数据源（akshare）	stock_board_industry_ths()	stock_board_concept_ths()
个股重叠	低（一只股票一个行业）	高（一只股票多个概念）

选择概念板块的原因： 概念板块的涨跌直接反映市场对这个叙事（逻辑）的态度，更适合 L2 层面的追踪。

1.3 数据架构重构背景

当前 DATA_DIR 下 30+ 个 JSON 文件平铺，包括：

data/3l/

all_stocks_60d.json

（个股K线）

sector_daily.json

（行业板块K线）

stock_industry_map.json

（个股→行业映射）

mainline_history.json

（主线持续天数）

plan_tracking.db

（操作计划追踪SQLite）

private/

（用户数据）

cache/

（临时缓存）

charts/

（SVG图表）

...

（25+ 其他文件平铺）

随着概念板块追踪、板块波谷轮动等新功能加入，需要更清晰的数据分层。

2. 数据现状

2.1 已有数据

数据	路径	状态
个股K线（60天）	all_stocks_60d.json	✔ 有（1.8MB）
行业板块K线	sector_daily.json	✔ 有（3.1MB）
个股→行业映射	stock_industry_map.json	✔ 有（4680条，含 ths_industry）
个股→沪深300/中证500	board_constituents.json	✔ 有
中证全指	index_sh_data.json	✔ 有

主线历史	mainline_history.json	✔ 有
自选股/方向	private/watchlist.json	✔ 有

2.2 缺失数据

数据	用途	来源	状态
个股→概念板块映射	自选股→概念板块关联	push2test f103	✗ 需新建
概念板块K线	概念板块波谷计算	akshare stock_board_concept_ths() 或 push2test 板块K线接口	✗ 需新建
概念板块成分股	板块包含哪些股票	push2test b:BKxxx	✗ 需新建
核心概念池	用户需要追踪的概念列表	从自选股映射筛选	✗ 需新建

2.3 已有接口能力

```
# push2test 个股行情 → f103 返回概念板块列表（逗号分隔）
GET https://push2test.eastmoney.com/api/qt/clist/get
  params: fs=m:0+t:6+f:!2,m:0+t:80+f:!2,m:1+t:2+f:!2,m:1+t:23+f:!2
  fields=f12,f14,f103
→ 每个股票返回所属概念板块（如 "AI算力,机器人,国产替代"）

# push2test 概念板块成分股
GET https://push2test.eastmoney.com/api/qt/clist/get
  params: fs=b:BKxxx+f:!50
  fields=f12,f14
→ 返回板块内成分股列表
```

3. 数据架构重构方案

3.1 分层目录结构

将平铺的 JSON 文件按用途分四层：

```
data/3l/
|
├─ raw/                                [只由 cron 写入]
|   ├── stocks/                        个股K线（原 all_stocks_60d.json）
|   ├── sectors/                       行业板块K线（原 sector_daily.json）
|   └── concepts/                       概念板块K线（新增）
|       └── concept_daily.json
|
├─ map/                                [只由 update_stock_data 全量更新]
|   ├── stock_industry.json            个股→行业映射（原 stock_industry_map.json）
|   ├── stock_concept.json             个股→概念映射（新增）
|   ├── concept_list.json              概念板块列表 + 成分股（新增）
|   └── stock_meta.json                个股基础信息（code→name 映射等）
|
├─ track/                              [由 service 层读写]
|   ├── plan_tracking.db               操作计划追踪 SQLite
|   ├── concept_wave.db                概念板块波谷追踪 SQLite（新增）
|   └── mainline_history.json           主线持续天数
|
├─ cache/                              [临时缓存，可清]
|   └── .cache/                        各模块缓存
|
├─ private/                            [用户数据]
|   ├── watchlist.json                 自选股
|   ├── holdings.json                  持仓
|   └── directions.json                自选方向
|
├─ public/                             [对外公开]
|   └── charts/                        SVG 图表
|
```

3.2 迁移策略

原则：向后兼容，不破坏已有路径引用。 - 旧文件保留访问（`config.py` 中 `ALL_STOCKS_PATH` 等路径常量不变） - 新增函数优先写新路径 - 逐步将读取逻辑指向新路径 - `migration` 脚本在功能完成后统一执行

3.3 概念板块 SQLite 表结构

```
-- track/concept_wave.db

-- ① 概念板块基础信息
CREATE TABLE concepts (
  code      TEXT PRIMARY KEY, -- 'BK1682'
  name      TEXT NOT NULL,    -- 'AI算力'
  stock_count INTEGER,        -- 成分股数量
  created_at TEXT,
  updated_at TEXT
);

-- ② 用户追踪的概念板块
CREATE TABLE tracked_concepts (
  code      TEXT PRIMARY KEY,
  name      TEXT,
  source    TEXT, -- 'from_watchlist' / 'manual' / 'scan_hot'
  added_at  TEXT,
  is_active INTEGER DEFAULT 1 -- 0=暂停追踪
);

-- ③ 概念板块每日波谷追踪记录
CREATE TABLE concept_wave (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  code      TEXT NOT NULL, -- 概念板块代码
  date      TEXT NOT NULL, -- YYYY-MM-DD
  close     REAL, -- 当日收盘
  change_pct REAL, -- 当日涨跌幅
  ema5      REAL,
  ema10     REAL,
  ema20     REAL,
  ema60     REAL, -- 中长期均线
  volume_ratio REAL, -- 量比（当日量/20日均量）
  bias5     REAL, -- 乖离率5
  bias10    REAL,
  bias20    REAL,
  pk_score  INTEGER DEFAULT 0, -- 偏波峰评分（复用小波峰波谷逻辑）
  vl_score  INTEGER DEFAULT 0, -- 偏波谷评分
  stage     TEXT, -- '波谷' / '波中' / '波峰' / '下跌' / '上升'
  is_wave_bottom INTEGER DEFAULT 0, -- 标志：是否可能处于波谷
  last_peak_date TEXT, -- 最近波峰日期
  last_trough_date TEXT, -- 最近波谷日期
  UNIQUE(code, date)
);

-- ④ 追踪建议（当概念板块出现波谷信号时生成）
CREATE TABLE wave_alerts (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  code      TEXT NOT NULL, -- 概念板块代码
  date      TEXT NOT NULL, -- 信号日期
  wave_type TEXT, -- 'valley' / 'peak'
  confidence INTEGER, -- 置信度 1-5
  reason    TEXT, -- 触发理由
  notified  INTEGER DEFAULT 0, -- 是否已推送到微信
  created_at TEXT
);
```

4. 概念板块波谷追踪系统设计

4.1 概念板块筛选（三层）

第一层：核心追踪（持续盯）

条件：自选股涉及的概念板块

操作：每天拉 K 线，计算波谷位置

预期数量：50~80 个

第二层：扫描观察（低频率）

条件：全市场概念板块 20 日涨幅排名

操作：每天扫排名，发现不在核心池但持续向上的新概念

预期数量：关注 5~10 个异常信号

第三层：剔除

条件：与交易风格无关（白酒/地产/银行/煤炭等）

操作：完全不追踪

自选股→概念板块映射流程：

```
def build_tracked_concepts(watchlist_codes):  
    """从自选股提取核心概念板块"""  
    # 1. 读 stock_concept.json (个股→概念映射)  
    # 2. 找到自选股涉及的所有概念板块  
    # 3. 按出现频率排序（覆盖面高=更相关）  
    # 4. 写入 tracked_concepts 表
```

4.2 概念板块波谷识别规则

参照大盘 V5 波峰波谷评分法的思路，适配到概念板块级别：

条件1：价格远离EMA20 (BIAS20 < -5%) → 可能波谷

条件2：连续下跌后成交量萎缩（量比 < 0.7）→ 供应枯竭

条件3：EMA10 斜率从负转平或转正 → 趋势可能见底

条件4：近 N 日跌幅达到该板块2σ水平 → 极端价格

输出：vl_score (0~4)

vl_score >= 3 → 偏波谷信号，提示关注

vl_score >= 4 → 强波谷信号，建议检查是否可切入

与大盘波峰波谷的关系：- 大盘偏波谷时 → 所有板块的波谷置信度加权 + 0.5 - 大盘偏波峰时 → 弱势板块的波谷信号可能只是下跌中继，加权 - 0.5 - 概念板块独立走强（大盘跌它横盘）→ 轮动启动迹象，加权 + 1.0

4.3 数据流

cron 17:00 update_stock_data.py

└─ update_all_stocks() → raw/stocks/ 个股K线

└─ update_sectors() → raw/sectors/ 行业板块K线

└─ update_concept_maps() [新增] → map/stock_concept.json、map/concept_list.json

└─ update_concept_klines() [新增] → raw/concepts/ 概念板块K线

页面首次加载（概念波谷追踪页）

└─ 从 track/concept_wave.db 读取

└─ 计算波谷阶段

└─ 信号推送（如果有强波谷信号）

└─ 缓存计算结果

用户手动触发

└─ POST /api/concept-wave/refresh

└─ 强制重新计算

4.4 API 设计

```
GET /api/concept-wave  
返回：  
{  
    "tracked": [{code, name, stage, vl_score, is_wave_bottom, ...}],
```

```
"alerts": [{code, name, wave_type, confidence, reason}],
"new_hot": [{code, name, chg_20d}] // 扫描层发现的新概念
}

GET /api/concept-wave/detail?code=BKxxxx
    返回单个概念板块的波谷趋势详情

POST /api/concept-wave/add?code=BKxxxx
    手动添加追踪概念

POST /api/concept-wave/remove?code=BKxxxx
    移除追踪概念
```

4.5 前端页面

新增独立页面 `ConceptWaveTracking.tsx` , 路径 `/concept-wave.html`

页面结构：

概念板块波谷追踪

【日期筛选】【刷新】

=== 核心追踪概念 (N个) ===

概念	阶段	vL_score	近5日涨跌
AI算力	波谷	4	-1.2%
机器人	波中	1	+0.5%
HBM概念	下跌	2	-3.1%
...			

=== 波谷告警 ===

▲ AI算力 vL_score=4, 缩量+BIAS20=-6.2%, 关注切入机会

=== 新概念扫描 ===

🔍 时空大数据 (BKxxxx) 20日涨幅 +18%, 不在核心关注列表

5. 实现计划

阶段一：数据基建 (P0)

#	任务	预估
1	update_stock_data.py: 新增 <code>update_concept_maps()</code> → 从 push2test 拉取个股→概念映射	1h
2	update_stock_data.py: 新增 <code>update_concept_klines()</code> → 拉概念板块K线	1.5h
3	概念板块波谷 SQLite 表结构 + <code>_init_db()</code>	0.5h
4	数据层 data_layer 新增概念板块读取函数	0.5h

阶段二：波谷判断逻辑 (P0)

#	任务	预估
5	概念板块波谷评分函数 <code>judge_concept_wave()</code>	1.5h
6	波谷信号检测 + 告警生成	1h
7	单元测试 (数据映射/波谷评分/告警逻辑)	1h

阶段三：API + 前端 (P1)

#	任务	预估

8	后端 API 路由 <code>/api/concept-wave</code>	0.5h
9	前端页面 <code>ConceptWaveTracking.tsx</code>	1.5h
10	微信推送集成（强波谷信号推送到手机）	1h

阶段四：数据架构重构 (P2)

#	任务	预估
11	新目录结构实现 + 迁移脚本	1h
12	<code>config.py</code> 路径常量统一	0.5h
13	全回归 + 验证生产数据一致性	1h

6. 不做 (v1 范围外)

项目	原因
概念板块内部轮动分析	复杂度高，v2 再考虑
概念板块热度排行可视化	表格足够，v2 再加图
自动建仓建议	只做信号提示，不做自动决策
全量 486 个概念 K 线实时同步	只拉用户相关概念，降低数据量
行业板块波谷追踪同步改造	概念板块先做，稳定后再扩展

7. 文件清单

新增：	
<code>docs/concept-wave-tracking-design.md</code>	- 本设计文档
<code>server/backend/services/concept_wave_service.py</code>	- 概念波谷追踪服务
<code>server/backend/api/concept_wave.py</code>	- API 路由
<code>server/backend/tests/test_concept_wave.py</code>	- 测试
<code>server/frontend/src/pages/ConceptWaveTracking.tsx</code>	- 前端页面
<code>server/frontend/src/pages/ConceptWaveTracking.css</code>	- 样式
修改：	
<code>server/backend/core/update_stock_data.py</code>	- 新增概念板块数据更新
<code>server/backend/core/data_layer.py</code>	- 新增概念板块读取函数
<code>server/server.py</code>	- 注册路由
数据：	
<code>data/3l/map/stock_concept.json</code>	- 个股~概念映射
<code>data/3l/map/concept_list.json</code>	- 概念板块列表
<code>data/3l/raw/concepts/concept_daily.json</code>	- 概念板块K线
<code>data/3l/track/concept_wave.db</code>	- 波谷追踪SQLite

8. 分支策略

- 分支名: `feature/concept-wave-tracking`
- 基于 master 创建
- TDD 开发
- 完成后 PR 合并